راهنمای تمرین مصاحبه

پروژه‌ی `erc4626-donation-attack-lab`

توضیح خط‌به‌خط، به زبان ساده

برای: حسام قربانی | تاریخ: 28 اکتبر 2026

این فایل برای **یاد گرفتنه**، نه برای حفظ کردن. هر چیزی که اینجا نوشته شده رو باید بتونی با زبان خودت بگی. اگه جایی رو نفهمیدی، همون‌جا علامت بزن و از Claude بپرس.

بخش 0: چطور از این فایل استفاده کنم؟

1. اول بخش 1 تا 3 رو بخون. این‌ها «داستان» پروژه‌ست. هنوز سراغ کد نرو.
2. بعد کد رو توی VS Code باز کن و بخش 4 رو کنارش بخون. هر تکه کد، زیرش توضیح ساده‌ش هست.
3. بخش 7 (سؤال‌های مصاحبه) رو **بلند** بخون و جواب بده. جواب‌ها به انگلیسی هم هست چون مصاحبه‌ها معمولاً انگلیسیه.
4. بخش 8 (تمرین‌ها) رو انجام بده: کد رو خراب کن، قبلش حدس بزن چی می‌شه، بعد تست رو اجرا کن و ببین.

قانون طلایی: اگه چیزی رو نتونی با زبان خودت و بدون نگاه کردن بگی، یعنی هنوز یاد نگرفتی. این اشکالی نداره؛ فقط یعنی باید دوباره بخونی یا بپرسی.

نکته‌ی صداقت: این پروژه با کمک AI ساخته شده. سر مصاحبه همین رو بگو و بعد نشون بده که چقدر ازش فهمیدی. این از «خودم نوشتم» خیلی محکم‌تره، چون راسته و قابل چک کردنه.

بخش 1: این پروژه چیه؟ (جواب 30 ثانیه‌ای)

پروژه یه **آزمایشگاه کوچیکه**. سه تیکه داره:

1. یه «قلک مشترک» (`vault`: `VulnerableVault`) که **آسیب‌پذیره**.

2. یه **حمله** که از همون قلک پول می‌دزده، و ثابت شده با تست.

3. **درست‌شده**: یه قلک `SafeVault`. همون حمله رو می‌زنیم و این بار مهاجم ضرر می‌کنه.

اسم این حمله **donation attack** یا **inflation attack** (استاندارد قلک‌های ـه و روی استاندارد `ERC-4626` توکن) اتفاق می‌افته.

جمله‌ی آماده برای مصاحبه (انگلیسی)

```
"This is a small Foundry project about the ERC-4626 donation attack,
also called the inflation attack. It has a vulnerable vault, a test
that shows an attacker making a profit, and a fixed vault that uses
virtual shares and assets. I built it with the help of an AI assistant,
then I ran the tests and studied how it works. The vaults are
simplified and not audited."
```

معنی فارسی

«این یه پروژه‌ی کوچیک با Foundry درباره‌ی حمله‌ی donation روی ERC-4626 ـه. یه vault آسیب‌پذیر داره، یه تست که نشون می‌ده مهاجم سود می‌کنه، و یه vault درست‌شده که از سهم و دارایی مجازی استفاده می‌کنه. من با کمک یه دستیار AI ساختمش، بعد تست‌ها رو اجرا کردم و یاد گرفتم چطور کار می‌کنه. vault ها ساده شدن و audit نشدن.»

بخش 2: واژه‌نامه (کلمه‌های مهم)

کلمه	معنی ساده
`Solidity`	زبان برنامه‌نویسی قراردادهای هوشمند روی اتریوم.
`smart contract`	برنامه‌ای که روی بلاکچین می‌مونه و هر کی می‌تونه صداش کنه. بعد از انتشار، تغییرش سخته.
`token` / `ERC-20`	استانداردیه که می‌گه هر توکن چه تابع‌هایی داشته باشه (`transfer`، `balanceOf`، ...). یه پول دیجیتال.
`vault`	یه قلک مشترک. آدم‌ها توکن می‌ریزن و در عوض «سهم» می‌گیرن.
`share` (سهم)	مثل کوپن. نشون می‌ده چند درصد از قلک مال توئه.
`ERC-4626`	استانداردی برای قلک‌های توکن‌دار: `deposit`، (واریز) و `redeem` (برداشت با سهم) ...
`totalAssets`	کل توکنی که الان تو قلکه.
`totalShares`	کل کوپن‌هایی که صادر شده.
قیمت هر سهم	`totalAssets ÷ totalShares`. هر کوپن چند توکن می‌ارزه.
`donation` (اهدا)	`deposit` رو صدا بزنی. `transfer`)، بدون اینکه فرستادن توکن **مستقیم** به قلک (با پول بالا می‌ره ولی کوپنی صادر نمی‌شه.
`rounding` گرد شدن به پایین (`down`)	`7 ÷ 2 = 3`، یعنی .5 دور ریخته می‌شه .اعشار نداره Solidity تقسیم عدد صحیح در.
`virtual shares/assets`	سهم و دارایی «مجازی» که فقط تو فرمول جمع می‌شن، تا قیمت رو نشه دستکاری کرد.
`wei`	`1` یعنی یه ذره‌ی `1e18` یک توکن کامله و `1 توکن = 10^18 wei`. یعنی :کوچک‌ترین واحد ناچیز.
`uint256`	عدد صحیح بدون علامت (فقط 0 و مثبت‌ها). تا 256 بیت.
`address`	آدرس یه کیف پول یا یه قرارداد.

`mapping`	مثل دفترچه‌ی تلفن: کلید ← مقدار. مثلاً آدرس ← تعداد سهم.
`msg.sender`	کسی که الان تابع رو صدا زده.
`view`	تابعی که فقط می‌خونه و چیزی رو عوض نمی‌کنه.
`external` / `public` / `internal`	کی می‌تونه تابع رو صدا بزنه: از بیرون / از همه‌جا / فقط از داخل.
`immutable`	`constructor`. تعیین می‌شه و بعدش ثابته مقداری که فقط تو
`constant`	مقدار ثابتی که موقع نوشتن کد معلومه.
`revert` / `error`	`error ZeroShares()`. یه پیام خطای سفارشیه .کل تراکنش رو لغو می‌کنه
`Foundry` / `forge`	`forge test`. تست‌ها رو اجرا می‌کنه .Solidity ابزار تست و ساخت قراردادهای
`fuzz test`	یه تست که با کلی عدد **تصادفی** اجرا می‌شه (اینجا 1000 بار).
`invariant`	قانونی که همیشه باید درست بمونه، مهم نیست چه کارهایی انجام بشه. (این پروژه هنوز invariant test نداره.)
`vm.prank`	تو تست، وانمود می‌کنه یه آدرس دیگه تابع رو صدا زده.
`assert...`	تو تست چک می‌کنه یه چیزی درسته. اگه نباشه، تست شکست می‌خوره.

بخش 3: داستان حمله، با عدد

مثال ساده (کوپن)

یه قلک مشترک داریم. هر کی پول بریزه، کوپن می‌گیره. قیمت هر کوپن برابره با پول قلک تقسیم بر تعداد کوپن‌ها. تقسیم‌ها گرد می‌شن **به پایین**.

یادآوری: `1` (`wei`). `1e18` یعنی یه توکن کامل (یک و هجده‌تا صفر).

مرحله‌به‌مرحله، روی `VulnerableVault`

قدم	چی می‌شه	توکن تو قلک	کل سهم‌ها
0	قلک خالیه.	0	0
1	`1 wei` واریز می‌کنه. چون قلک خالیه، مهاجم کوچک‌ترین ذره یک‌به‌یک سهم می‌گیره.	1	1 (مهاجم)
2	مهاجم یه توکن کامل رو **مستقیم** به قلک می‌فرسته (donation). هیچ سهمی صادر نمی‌شه.	1e18 + 1	1
3	قربانی نزدیک دو توکن واریز می‌کنه. سهمش 1.99999... می‌شه که گرد می‌شه به **یک**.	3e18	2
4	هر سهم حالا 1.5e18 می‌ارزه. سهم قربانی و سهم مهاجم هر کدوم 1.5e18.	3e18	2

حساب قدم 3 (برای دقت):

```
victim deposits V = 2e18 - 1
```

```
shares = V * totalShares / totalAssets
       = (2e18 - 1) * 1 / (1e18 + 1)
       = 1.99999...   ->  rounds down to 1
```

نتیجه: قربانی حدود 2 توکن ریخت ولی سهمش فقط 1.5 توکن می‌ارزه: حدود **نیم توکن ضرر**. مهاجم حدود 1 توکن خرج کرد و 1.5 توکن گرفت: حدود **نیم توکن سود**.

چرا کار می‌کنه؟ دو دلیل

1. **قیمت از `balanceOf` خونده می‌شه**. هر کسی می‌تونه توکن بفرسته به قلک و قیمت رو بالا ببره. (اسمش «دستکاری قیمت»ـه.)

2. **گرد شدن به پایین**. وقتی قیمت هر سهم خیلی بالاست، 1.99999 سهم می‌شه یک سهم. باقی‌مونده تو قلک می‌مونه و بین صاحب‌ها تقسیم می‌شه، یعنی بخشی از پول قربانی به مهاجم می‌رسه.

همین حمله روی `SafeVault`

فرمول جدید:

```
shares = assets * (totalShares + 1000) / (totalAssets + 1)
```

یعنی 1000 سهم و 1 توکنِ «مجازی» همیشه تو حساب هستن.

قدم	چی می‌شه	نتیجه
1	`1 wei` واریز می‌کنه. سهمش 1000 می‌شه (چون سهم مجازی مهاجم هست.)	مهاجم: 1000 سهم
2	مهاجم یه توکن کامل donate می‌کنه.	قلک: 1 + 1e18
3	قربانی نزدیک دو توکن واریز می‌کنه. سهمش 3999.99... می‌شه که گرد می‌شه به 3999.	قربانی: 3999 سهم
4	قلک 3e18 توکن و 4999 سهم داره. ارزش سهم‌ها با احتساب مجازی‌ها:	مهاجم ≈ 0.50 توکن / قربانی ≈ 1.9998 توکن

```
shares(victim) = (2e18 - 1) * (1000 + 1000) / (1e18 + 1 + 1)
```

```
               = 3999.99...  ->  rounds down to 3999
```

نتیجه: مهاجم حدود 1 توکن خرج کرد و فقط حدود نیم توکن گرفت: **حدود نیم توکن ضرر**. قربانی فقط حدود 0.0083 درصد از پولش رو از دست داد (خیلی کمتر از 0.1 درصد).

چرا؟ سهم‌های مجازی مثل یه صاحب نامرئی هستن که یه تیکه‌ی بزرگ از اون donation رو می‌برن. پس مهاجم نمی‌تونه همه‌ش رو پس بگیره.

بخش 4: کد، خط‌به‌خط

فایل‌ها رو توی VS Code کنار این PDF باز کن. هر قسمت کد رو بالا می‌بینی و توضیحش رو پایین.

4.1 فایل `src/IERC20Like.sol`

`interface`) که می‌گه یه توکن حداقل چه تابع‌هایی باید داشته باشه. خودش کاری نمی‌کنه) «یه «قالب.

```
// SPDX-License-Identifier: MIT
```

یه کامنت که نوع مجوز کد رو می‌گه (MIT = هر کی می‌تونه استفاده کنه). اگه نباشه، کامپایلر هشدار می‌ده.

```
pragma solidity ^0.8.24;
```

«^ یعنی «یا جدیدتر در همین خط .می‌گه این کد با کامپایلر نسخه‌ی 0.8.24 یا بالاتر (تا قبل از 0.9) کار می‌کنه.

```
interface IERC20Like {
```

`interface` هیچ تابعی بدنه نداره تعریف قالب. تو.

```
function balanceOf(address account) external view returns (uint256);
```

`view` یعنی فقط می‌خونه .«می‌پرسه «این آدرس چقدر توکن داره؟.

```
function transfer(address to, uint256 amount) external returns (bool);
```

`amount` `to` توکن می‌فرسته به ،از حساب صدازننده.

```
function transferFrom(address from, address to, uint256 amount) external returns (bool);
```

`approve`) داده باشه) `from` قبلاً اجازه `to` می‌ده. فقط اگه `from` برمی‌داره و به از حساب.

4.2 فایل `src/MockToken.sol`

`mint`). **هیچ‌وقت** تو پروژه‌ی واقعی) یه توکن **تقلبی و ساده** فقط برای تست. هر کسی می‌تونه ازش بسازه استفاده نکن.

```
contract MockToken {
```

شروع قرارداد.

```
uint256 public totalSupply;
```

`public` یعنی خودکار یه تابع خوندن هم می‌سازه .کل توکن‌های ساخته‌شده.

```
mapping(address => uint256) public balanceOf;
```

دفترچه: آدرس ← موجودی.

```
mapping(address => mapping(address => uint256)) public allowance;
```

دفترچه دوتایی: «صاحب» ← «کی اجازه داره» ← «تا چقدر». مثلاً من به قلک اجازه می‌دم تا 100 توکن از من برداره.

```
function mint(address to, uint256 amount) external {
    totalSupply += amount;
    balanceOf[to] += amount;
}
```

mint تو توکن واقعی هر کسی نمی‌تونه) .`to` رو توکن جدید می‌سازه: هم کل توکن‌ها رو زیاد می‌کنه، هم موجودی (.کنه

```
function approve(address spender, uint256 amount) external returns (bool) {
    allowance[msg.sender][spender] = amount;
    return true;
}
```

.`amount` از توکن‌هام رو خرج کنه `spender` اجازه می‌دم تا من (صدازننده) به

```
function transfer(address to, uint256 amount) external returns (bool) {
    balanceOf[msg.sender] -= amount;
    balanceOf[to] += amount;
    return true;
}
```

`revert` می‌کنه خودش Solidity 0.8، `to` اضافه می‌کنه. اگه موجودی کافی نباشه از موجودی من کم می‌کنه و به (چون عدد منفی نداریم).

```
function transferFrom(address from, address to, uint256 amount) external returns (bool) {
```

```
    uint256 allowed = allowance[from][msg.sender];

    if (allowed != type(uint256).max) {

        allowance[from][msg.sender] = allowed - amount;

    }

    balanceOf[from] -= amount;

    balanceOf[to] += amount;

    return true;

}
```

`from`. اول چک می‌کنه اجازه‌ی کافی هست و از اجازه کم می‌کنه. اگه اجازه `transfer` ولی از حساب مثل `type(uint256).max`) باشه، کم نمی‌کنه) ««نامحدود

Transfer.) و چک‌های یه توکن واقعی رو هم ندارد. فقط برای تسته event ندارد (مثل محدودیت: این توکن

4.3 فایل (مهم‌ترین فایل) `src/VulnerableVault.sol`

```
import {IERC20Like} from "./IERC20Like.sol";
```

قالب توکن رو از فایل کناری می‌آره.

```
contract VulnerableVault {
```

```
    error ZeroShares();
```

شروع قرارداد، و تعریف یه خطای سفارشی. اگه کسی واریز کنه و سهمش 0 بشه، تراکنش لغو می‌شه.

```
IERC20Like public immutable asset;
```

`immutable`. یعنی فقط یه‌بار موقع ساخت تنظیم می‌شه .آدرس توکنی که تو قلک نگه می‌داریم

```
uint256 public totalShares;
```

کل سهم‌هایی که صادر شده.

```
mapping(address => uint256) public sharesOf;
```

هر آدرس چند سهم داره.

```
constructor(IERC20Like _asset) {
```

```
    asset = _asset;
}
```

موقع ساخت قلک، توکن رو تنظیم می‌کنه.

```
function totalAssets() public view returns (uint256) {
```

```
    return asset.balanceOf(address(this));
}
```

`address(this)` یعنی خود قلک). چون از موجودی) **نقطه‌ی ضعف** 1. کل دارایی = موجودی واقعی توکنِ قلک واقعی می‌خونه، هر کسی که توکن بفرسته، این عدد رو بالا می‌بره.

```
function convertToShares(uint256 assets) public view returns (uint256) {
```

```
    uint256 supply = totalShares;
    return supply == 0 ? assets : (assets * supply) / totalAssets();
}
```

`supply == 0`)، 1:1 (هر توکن یه) می‌گه «با این مقدار توکن، چند سهم می‌گیرم؟». اگه هیچ سهمی نیست `توکن × کل سهم ÷ کل دارایی`. **نقطه‌ی ضعف** 2. تقسیم گرد می‌شه به پایین سهم). وگرنه.

```
function convertToAssets(uint256 shares) public view returns (uint256) {
```

```
    uint256 supply = totalShares;
    return supply == 0 ? shares : (shares * totalAssets()) / supply;
}
```

برعکسش: «این تعداد سهم چند توکن می‌ارزه؟». اینم گرد می‌شه به پایین (برای قلک امنه).

```
function deposit(uint256 assets) external returns (uint256 shares) {
```

```
    shares = convertToShares(assets); // before the transfer
    if (shares == 0) revert ZeroShares();
    asset.transferFrom(msg.sender, address(this), assets);
    totalShares += shares;
    sharesOf[msg.sender] += shares;
}
```

واریز. اول حساب می‌کنه چند سهم می‌گیری. **ترتیب مهمه**: اگه بعد از انتقال حساب کنه، موجودی قلک شامل واریز تو هم هست و سهم کمتر محاسبه می‌شه. بعد اگه سهم 0 بود لغو می‌کنه (که تو رو از باخت کامل نجات می‌ده). بعد توکن رو می‌گیره و سهم رو ثبت می‌کنه.

```
function redeem(uint256 shares) external returns (uint256 assets) {
```

```
    assets = convertToAssets(shares);

    sharesOf[msg.sender] -= shares;

    totalShares -= shares;

    asset.transfer(msg.sender, assets);
}
```

برداشت: سهم می‌ده و توکن می‌گیره. اول مقدار رو حساب می‌کنه، بعد سهم رو **کم می‌کنه** و آخر توکن رو می‌فرسته. (اول حساب‌ها رو به‌روز می‌کنه، بعد پول می‌فرسته؛ این الگوی امن‌تریه.)

محدودیت: `transferFrom` رو صدا می‌زنه و بعد حساب‌ها رو به‌روز می‌کنه. با توکن‌های عجیب (که موقع `deposit` اول مشکلی نیست انتقال کد اجرا می‌کنن) این می‌تونه خطرناک باشه. با `MockToken`.

4.4 فایل `src/SafeVault.sol` (چه چیزی فرق می‌کنه؟)

`VulnerableVault` ـه. فقط **فرمول‌های تبدیل** عوض شدن تقریباً همه‌چیز مثل.

```
uint256 private constant VIRTUAL_SHARES = 1000; // 10 ** 3
```

```
uint256 private constant VIRTUAL_ASSETS = 1;
```

`private` یعنی بیرون از قرارداد دیده نمی‌شه .دو عدد ثابت: 1000 سهم مجازی و 1 توکن مجازی.

```
function convertToShares(uint256 assets) public view returns (uint256) {
```

```
    return (assets * (totalShares + VIRTUAL_SHARES)) / (totalAssets() + VIRTUAL_ASSETS);
}
```

فرمول جدید. دیگه حالت «قلک خالیه» جدا نداره، چون همیشه 1000 سهم مجازی هست. و مخرج هرگز صفر نمی‌شه (چون +1).

```
function convertToAssets(uint256 shares) public view returns (uint256) {
```

```
    return (shares * (totalAssets() + VIRTUAL_ASSETS)) / (totalShares + VIRTUAL_SHARES);
}
```

فرمول برعکس. باز هم گرد می‌شه به پایین.

نکته: این فرمول مثل پیاده‌سازی OpenZeppelin یه تمرین برابر offset ـه. ولی من فقط با 1000 تست کردم. بخش 8 های دیگه offset داره برای.

4.5 فایل `test/DonationAttack.t.sol`

```
import {Test} from "forge-std/Test.sol";
```

کتابخونه‌ی تست Foundry. ابزارهایی مثل `Test` و `assertEq` می‌ده `vm`.

```
interface IVault {

    function deposit(uint256 assets) external returns (uint256);

    function sharesOf(address account) external view returns (uint256);

    function convertToAssets(uint256 shares) external view returns (uint256);

}
```

یه قالب مشترک برای هر دو قلک. چون هر دو همین تابع‌ها رو دارن، یه تابع کمکی می‌تونه هر دو رو حمله کنه.

```
uint256 internal constant DONATION = 1e18;

uint256 internal constant VICTIM_DEPOSIT = 2e18 - 1;
```

`2e18 - 1` واریز می‌کنه (دقیقاً زیر 2 برابر، تا گرد بشه به `1e18` 1 اهدا می‌کنه و قربانی عددهای داستان: مهاجم سهم).

```
address internal attacker = makeAddr("attacker");

address internal victim = makeAddr("victim");
```

`makeAddr` یه آدرس ساختگی با اون اسم می‌سازه (برای خوانایی تست).

```
function setUp() public {

    token = new MockToken();

}
```

قبل از هر تست اجرا می‌شه و یه توکن تازه می‌سازه.

```
token.mint(attacker, donation + 1);
```

```
token.mint(victim, victimDeposit);
```

به مهاجم و قربانی توکن می‌دیم تا بتونن واریز کنن.

```
vm.startPrank(attacker);
```

```
token.approve(address(vault), type(uint256).max);

vault.deposit(1);

token.transfer(address(vault), donation);

vm.stopPrank();
```

واریز wei همه‌چی رو مهاجم صدا می‌زنه». مهاجم اجازه می‌ده، 1 `startPrank` یعنی «از اینجا تا `stopPrank` رو **مستقیم** می‌فرسته (قدم 2) می‌کنه (قدم 1)، و `donation`.

```
vm.startPrank(victim);
```

```
token.approve(address(vault), type(uint256).max);

try vault.deposit(victimDeposit) {} catch {}

vm.stopPrank();
```

`try ... catch {}` یعنی «اگه لغو شد، اشکالی نداره، ادامه بده». لازمه چون تو .قربانی واریز می‌کنه (قدم 3) `SafeVault` گاهی واریز لغو می‌شه (سهم 0) و ما نمی‌خوایم تست خودش بترکه.

```
attackerSpent = donation + 1;
```

```
attackerValue = vault.convertToAssets(vault.sharesOf(attacker));

victimValue = vault.convertToAssets(vault.sharesOf(victim));
```

محاسبه‌ی نتیجه: مهاجم چقدر خرج کرد، و سهم هر کدوم الان چند توکن می‌ارزه.

سه تست

```
function test_vulnerableVault_attackerProfitsFromVictim() public {
```

```
    ...

    assertEq(vault.sharesOf(victim), 1, "...");

    assertGt(attackerValue, spent, "...");

    assertLt(victimValue, VICTIM_DEPOSIT, "...");

    assertGt(VICTIM_DEPOSIT - victimValue, 0.49e18, "...");
```

```
}
```

تست حمله. چک می‌کنه: قربانی دقیقاً 1 سهم گرفته، ارزش سهم مهاجم از خرجش بیشتره (سود)، ارزش سهم توکنه 0.49 از بیشتر قربانی ضرر و ،(ضرر) قربانی از واریزش کمتره (`assertGt`، `assertEq` = برابر). `assertLt` = کوچک‌تر، بزرگ‌تر.)

```
function test_safeVault_attackFails() public {
    ...
    assertLt(attackerValue, spent, "...");
    assertLt((VICTIM_DEPOSIT - victimValue) * 1000, VICTIM_DEPOSIT, "...");
}
```

`SafeVault`. چک می‌کنه مهاجم ضرر کرده. خط دوم: «ضرر قربانی × 1000 < **تست دفاع**. همون حمله روی واریز قربانی»، یعنی ضرر کمتر از یک‌هزارم (0.1%) واریزه.

```
function testFuzz_safeVault_attackerNeverProfits(uint256 donation, uint256 victimDeposit)
public {
    donation = bound(donation, 1, 1e30);
    victimDeposit = bound(victimDeposit, 1, 1e30);
    ...
    assertLe(attackerValue, spent);
}
```

تست fuzz. Foundry 1000 `donation` و جفت عدد تصادفی برای `victimDeposit`. `bound` امتحان می‌کنه `1e30` محدود می‌کنه. چک می‌کنه مهاجم **هیچ‌وقت** سود نکنه عدد رو به بازه‌ی 1 تا `assertLe` = کوچک‌تر یا) مساوی).

> این تست **اثبات نیست**. فقط می‌گه تو 1000 حالت تصادفی از همین یه نوع حمله، مهاجم سود نکرد.

4.6 فایل‌های تنظیمات و بقیه

فایل	چی کار می‌کنه
`foundry.toml`	تنظیمات: پوشه‌ی کد (`src`)، تعداد اجرای fuzz (`runs = 1000`)، نسخه‌ی کامپایلر (`0.8.24`).
`.gitignore`	به git نگه نداره و :می‌گه کدوم پوشه‌ها رو (خروجی‌های ساخته‌شده) `out/` و (`cache/`.

`.gitmodules` و پوشه‌ی `lib/forge-std`	نگه می‌داره. وقتی (submodule) «`forge-std`» رو به‌عنوان «زیرپروژه کتابخونه‌ی کلون کنه، خودش میاد کسی با `--recurse-submodules`.
`foundry.lock`	نسخه‌ی دقیق وابستگی‌ها رو قفل می‌کنه تا همه همون نسخه رو داشته باشن.
`LICENSE`	مجوز MIT: هر کی می‌تونه از کد استفاده کنه، ولی بدون ضمانت.
`README.md`	صفحه‌ی اول ریپو: پروژه چیه، چطور اجرا می‌شه، محدودیت‌ها چیه.
`.github/workflows/test.yml`	دستورالعمل تست خودکار (CI). هر `push` باعث می‌شه گیت‌هاب `forge test` رو اجرا کنه.

فایل CI، خط‌به‌خط

```
on: [push, pull_request]
```

`push` (بده، اجرا شو) کنه یا درخواست تغییر هر وقت کسی کد رو `pull request`.

```
runs-on: ubuntu-latest
```

روی یه کامپیوتر مجازی اوبونتو اجرا شو.

```
- uses: actions/checkout@v4
```

```
  with:
    submodules: recursive
```

کد رو دانلود کن، همراه با زیرپروژه‌ها (`forge-std`).

```
- uses: foundry-rs/foundry-toolchain@v1
```

Foundry رو نصب کن.

```
- run: forge test -vv
```

تست‌ها رو اجرا کن. اگه یکیش شکست بخوره، تیک قرمز می‌شه.

بخش 5: راه‌حل‌های دیگه‌ی همین مشکل

`SafeVault`: فقط **یه** راه‌حله. چیزهای دیگه‌ای هم هست که تو مصاحبه باید بدونی

راه‌حل	ایده	مشکلش
سهم و دارایی مجازی (مال ما)	یه offset تو فرمول، تا قیمت رو نشه خیلی دستکاری کرد.	بخشی از ارزش به سهم‌های «مجازی» می‌رسه و گردکردن هنوز کمی ضرر می‌ده.
سهم مرده (`dead shares`)	موقع اولین واریز، چندتا سهم به یه آدرس مرده می‌ریزی که هیچ‌وقت برداشته نشه. (Uniswap V2 همین کار رو می‌کنه.)	اولین واریزکننده یه مقدار کوچیک رو از دست می‌ده.
حسابداری داخلی	`totalAssets` نگه `balanceOf`، یه متغیر داخلی برای به جای `redeem` عوض می‌شه. پس/`deposit` می‌داری که فقط تو donation قیمت رو تغییر نمی‌ده.	توکن‌های اهدایی تو قلک گیر می‌کنن. و باید مراقب باشی.
حداقل واریز	واریز خیلی کوچیک (مثل 1 wei) رو قبول نمی‌کنی.	تنها کافی نیست.
چک حداقل سهم (`slippage`)	کاربر موقع واریز می‌گه «حداقل X سهم می‌خوام» و اگه کمتر شد لغو می‌شه.	مسئولیتش با کاربره/کیف پوله.

بخش 6: محدودیت‌های پروژه (صادقانه)

این‌ها رو **خودت** بگو. قبل از اینکه ازت بپرسن. این نشون می‌ده فهمیدی کجای کار ضعیفه:

- `vault` ها ساده‌شده‌ن و **کامل نیستن ERC-4626**: `preview*`، `maxDeposit`، `maxMint`، و `mint` `withd` ندارن `raw`.
  - تست fuzz فقط **یه نوع حمله** رو چک می‌کنه (مهاجم با 1 wei شروع می‌کنه و donate می‌کنه).
  - **invariant test** نداریم. (برنامه‌ریزی شده.)
- `MockToken`: ساده‌ست یا توکن‌های دارای fee-on-transfer مثل) `event` نداره و رفتار توکن‌های واقعی رو نداره (hook).
- `transferFrom`) می‌زنه. با توکن‌های) `deposit` قبل از به‌روز کردن حساب‌ها یه تماس بیرونی `nonReentrant` یا ترتیب دیگه‌ای بذاریم غیرقابل‌اعتماد باید.
  - پروژه **audit نشده** و برای استفاده‌ی واقعی نیست.
  - با کمک AI ساخته شده؛ من تست‌ها رو اجرا کردم و دارم یاد می‌گیرم.

بخش 7: سؤال‌های مصاحبه و جواب‌ها

هر سؤال: انگلیسی، جواب پیشنهادی انگلیسی (کوتاه)، و توضیح فارسی. **جواب رو با کلمات خودت بگو**، نه حفظی.

سؤال 1

What is an ERC-4626 vault?

It is a standard for tokenized vaults. You deposit an underlying token and receive shares. Shares represent a proportional claim on the vault's assets. The standard defines deposit, mint, withdraw, redeem and view helpers like convertToShares.

ERC-4626 یه استاندارد برای قلک‌هاست. توکن می‌ریزی، سهم می‌گیری. سهم یعنی نسبتی از کل قلک.

سؤال 2

Explain the donation (inflation) attack.

The attacker deposits 1 wei and gets 1 share. Then they transfer tokens directly to the vault, which raises the share price without minting shares. When a victim deposits, integer division rounds their shares down, so they get far fewer shares than their deposit is worth. The attacker redeems and keeps part of the victim's money.

مهاجم 1 wei واریز می‌کنه، بعد مستقیم توکن می‌فرسته تا قیمت سهم بالا بره. قربانی که واریز می‌کنه، به خاطر گرد شدن به پایین سهم خیلی کمتری می‌گیره و مهاجم بخشی از پولش رو می‌برد.

سؤال 3

Why does the victim only receive 1 share in your test?

Shares = assets * totalShares / totalAssets, with integer division. After the donation, totalAssets is about 1e18 and totalShares is 1. The victim deposits 2e18 - 1, which gives 1.99999 shares, and Solidity rounds that down to 1.

`shares = assets × totalShares ÷ totalAssets` و تقسیم صحیح گرد می‌شه به پایین. 1.99999 چون می‌شه 1.

سؤال 4

Why does the donation not give the attacker shares?

Because it is a plain token transfer. It does not call deposit(), so totalShares does not change, but the vault's token balance grows.

`deposit` رو صدا نمی‌زنه. فقط موجودی بالا می‌ره، سهمی صادر نمی‌شه `transfer` ساده‌ست و چون.

سؤال 5

Why is using balanceOf for totalAssets a problem?

Anyone can change a contract's token balance by sending tokens to it, so anyone can influence the share price. An alternative is internal accounting: a variable updated only in deposit and redeem.

`deposit/re` چون هر کسی می‌تونه توکن بفرسته و قیمت رو دستکاری کنه. راه دیگه: یه متغیر داخلی که فقط تو `deem` عوض می‌شه.

سؤال 6

How do virtual shares and assets help?

They add a fixed amount of virtual shares (1000) and virtual assets (1) to the conversion formulas. The donation is partly absorbed by the virtual shares, so the attacker cannot get all of it back, and the victim's rounding loss becomes tiny. In my test the attacker lost about half of the donation and the victim lost less than 0.1%.

سهم‌های مجازی یه تیکه از donation رو جذب می‌کنن، پس مهاجم همه‌ش رو پس نمی‌گیره. تو تست ما مهاجم حدود نصف رو از دست داد و ضرر قربانی کمتر از 0.1% بود.

سؤال 7

What does the ZeroShares check do? Is it enough?

It reverts when a deposit would mint zero shares, so a victim cannot lose their whole deposit for nothing. It is not enough: my exploit test shows the victim can still lose about half of the deposit.

جلوی باخت کامل رو می‌گیره ولی کافی نیست. تست ما نشون می‌ده قربانی هنوز تقریباً نصف رو از دست می‌ده.

سؤال 8

Why do you compute shares before transferFrom in deposit?

convertToShares reads totalAssets, which is the token balance. After the transfer, the balance already includes the new deposit, so the price would be wrong and the depositor would get too few shares.

چون موجودی قلک بعد از انتقال شامل واریز خودته و سهم کمتر محاسبه می‌شه.

سؤال 9

Which way should rounding go?

In favor of the vault. Deposit rounds shares down, redeem rounds assets down. For withdraw, shares burned should round up, and for mint the assets required should round up. My simplified vaults only have deposit and redeem.

`mint` `withdraw` و) `redeem`): توکن گرد به پایین) همیشه به نفع قلک. واریز: سهم گرد به پایین. برداشت با سهم (.ما نداریم؛ اونا گرد به بالا می‌خوان

سؤال 10

What does your fuzz test prove, and what does it not prove?

It checks that for 1000 random pairs of donation and victim deposit in the range 1 to 1e30, the attacker never ends up ahead in this specific attack. It does not prove the vault is safe: other attack shapes, other functions and real token behaviors are not covered.

نشون می‌ده تو 1000 حالت تصادفی از همین یه حمله مهاجم سود نکرده. اثبات امنیت نیست.

سؤال 11

What invariant tests would you add?

For example: the vault must stay solvent, meaning total redeemable assets are never more than the token balance; and nobody should be able to redeem more than they deposited unless someone donated. I would write a handler that can also donate, so the fuzzer explores the attack paths.

مثلاً: قلک همیشه باید توان پرداخت داشته باشه؛ و هیچ‌کس نباید بیشتر از واریزش برداشت کنه مگر اینکه کسی

donate کرده باشه. و یه handler که donate هم داشته باشه.

سؤال 12

What is the trade-off of the virtual offset size?

I have only tested an offset of 1000 so far. I plan to compare other values and the OpenZeppelin implementation.

راستش رو بگو: فقط با 1000 تست کردی. (تمرین 1 تو بخش 8 همین رو بررسی می‌کنه.)

سؤال 13

Could this vault be attacked with reentrancy?

With my mock token there are no hooks, so no. But deposit does an external call before updating state. With a token that has transfer hooks I would use a reentrancy guard or update state first.

باید `deposit` قبل از به‌روز کردن حساب‌ها تماس بیرونی می‌زنه. با توکن‌های دارای hook نه، ولی با `MockToken` یا ترتیب دیگه‌ای بذاریم `nonReentrant`.

سؤال 14

Did you write this yourself?

I built it with the help of an AI assistant. I ran the tests, I studied each file, and I can explain how the attack and the fix work. I am now practicing by changing the code and predicting what the tests will do.

راست رو بگو. هر چقدر واقعاً نوشتی یا عوض کردی همون رو بگو. هیچ‌وقت نگو «خودم نوشتم» اگه راست نیست. این جمله فقط وقتی کاملاً درسته که واقعاً تمرین‌های بخش 8 رو انجام داده باشی.

سؤال 15

What would you do next with this project?

Add invariant tests with a handler that can donate, compare my vault with OpenZeppelin's ERC4626, add the missing ERC-4626 functions, and try other attack paths.

invariant test، مقایسه با OpenZeppelin، تکمیل توابع ERC-4626 و تست حمله‌های دیگه.

بخش 8: تمرین‌های عملی

`forge test -vv` رو اجرا کن، و ببین حدست درست بود یا برای هر تمرین: **اول حدس بزن** چی می‌شه، **بعد** (یا `git restore .` Ctrl+Z) نه. بعد از هر تمرین، تغییرت رو برگردون.

تمرین 1: offset رو کم کن

1. بعد تست‌ها رو اجرا کن `VIRTUAL_SHARES = 1000` رو بکن `SafeVault.sol` عدد توی

•حدس بزن: کدوم تست قرمز می‌شه؟

- `1`: مهاجم هنوز ضرر می‌کنه، ولی قربانی حدود 10% از پولش رو از دست می‌ده. پس برابر offset با `test_safeVault_attackFails` باید به خاطر شرط «کمتر از 0.1%» شکست بخوره تست.
- `1000`%: حدود 0.0083 `10`: ضرر قربانی حدود 0.85%. با برابر offset با

•نتیجه‌ی یاد گرفتنی: offset بزرگ‌تر یعنی ضرر قربانی کمتر.

تمرین 2: ZeroShares رو بردار

`if (shares == 0) revert ZeroShares();` رو کامنت کن. یه تست جدید `VulnerableVault.sol` خط توی `1e18` واریز می‌کنه می‌کنه، قربانی donate `1e18`، واریز می‌کنه wei بنویس: مهاجم 1.

•حدس بزن: قربانی چند سهم می‌گیره؟

- `1e18` ریخته و هیچ چیزی نمی‌گیره، و مهاجم همه‌ی پولش رو می‌بره. این نشون جواب: 0 **سهم**. قربانی `ZeroShares` مهمه می‌ده چرا.

`deposit` رو عوض کن تمرین 3: ترتیب

`shares = convertToShares(assets);` `asset.transferFrom` رو **بعد** از `VulnerableVault.deposit` خط تو `(...)` بذار.

•حدس بزن: چی می‌شه؟

- `depos`، بعد از انتقال، موجودی قلک شامل واریز قربانی هم هست، پس سهم قربانی صفر حساب می‌شه `assertEq(... 1)`. باید شکست بخوره (`it` لغو می‌شه و تست اول

تمرین 4: همه‌چیز رو با کاغذ بکش

بدون نگاه کردن به PDF، جدول بخش 3 (قدم 1 تا 4) رو روی کاغذ بکش و عدد بذار. بعد با PDF مقایسه کن.

تمرین 5: هر فایل رو تو 1 دقیقه توضیح بده

بلند و بدون نگاه کردن، هر فایل رو برای یه آدم فرضی توضیح بده. اگه نتونستی، همون‌جا بخون و دوباره امتحان کن.

بعد از هر تمرین، اگه چیزی نفهمیدی یا جواب با حدست فرق داشت، یادداشت کن و از Claude بپرس. اشتباه حدس زدن بخش طبیعی یادگیریه.

بخش 9: قدم‌های بعدی برای یادگیری

- دوره‌ی Cyfrin Updraft (رایگان): هر بخشی که تموم کردی رو در README ثبت کن.
- Ethernaut و Damn Vulnerable DeFi: چند چالش حل کن و راه‌حلت رو با توضیح بنویس.
- همین پروژه: invariant test با handler که `donate` دارد، و مقایسه با که `ERC4626` OpenZeppelin.
- مسابقه‌های audit: اول فقط گزارش‌های قبلی رو بخون تا ببینی باگ‌ها چطور توصیف می‌شن.

هر چیزی که تو README بنویس باید **واقعاً انجام شده** باشه. تا انجام نشده، `Planning to` یا لینکدین می‌نویسی باید.